

Image Scaling Implemented in the JPEG-2000 Decoding Process Using Low Memory Algorithm by Aviv Kfir

The project presents an algorithm implementation of image magnification (interpolation) using the decoding system of JPEG-2000 standard. The Algorithm presented is using the same filters that are currently used in the standard, while using low memory buffers for the inverse DWT. The Algorithm is useful in the printing industry applications where scaling is needed and low memory is essential.

1. JPEG-2000 standard brief description

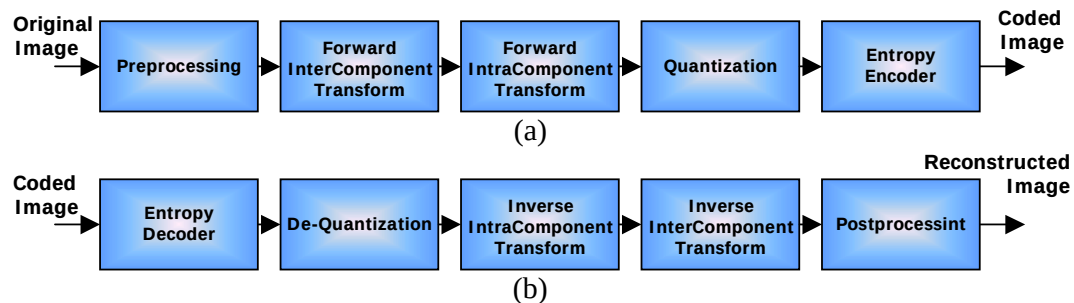


Fig 1: Codec structure. Structure of (a) encoder and (b) decoder

Preprocessing and postprocessing – The preprocessing stage centers the nominal dynamic range of the sampled data about zero. For example if sampled data are $\{0 : 2^P - 1\}$ then each sample is subtracted value of 2^{P-1} . The postprocessing returns the samples to their original dynamic range.

Forward and inverse intercomponent transform – This transform operates on all of the components together, and serves to reduce the correlation between components, leading to improved coding efficiency. The forward transform maps image data from RGB to YCrCb color space, and then each component is treated independently. The inverse transform return image to RGB color space.

Forward and inverse intracomponent transform – In this stage, transforms that operate on individual components can be applied. The operator that employed is the wavelet transform. Through the application of the wavelet transform, a component is split into numerous frequency bands (i.e., sub-bands). Due to the statistical properties of these sub-bands signals, the transformed data can usually be coded more efficiently than the original untransformed data.

The basic building block for such transforms is the 1-D 2-channel perfect-reconstruction (PR) uniformly-maximally-decimated (UMD) filter-bank (FB) which has the general form shown in Fig 2. The analysis side of the UMDFB, depicted in Fig 2(a), is associated with the forward transform, while the synthesis side, depicted in Fig 2(b), is associated with the inverse transform.

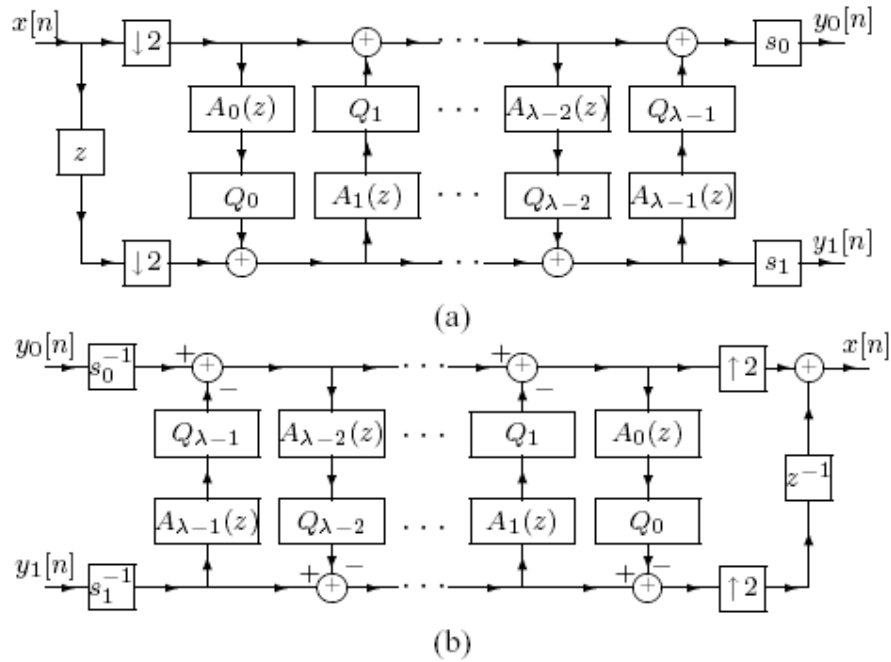


Fig 2: Lifting realization of 1-D 2-channel PR UMDFB. (a) Analysis side. (b) Synthesis side.

Since and image is a 2-D signal, clearly we need a 2-D UMDFB. By applying the 1-D UMDFB in both the horizontal and vertical directions, a 2-D UMDFB is effectively obtained. The wavelet transform is then calculated by recursively applying the 2-D UMDFB to the low-pass sub-band signal obtained at each level in the decomposition. The result of the decomposition is illustrated in Fig 3.

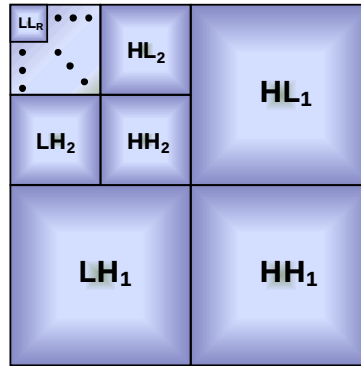


Fig 3: Subband structure

Quantization and dequantization – After intracomponent transform is applied, the resulting coefficients are quantized. Quantization allows greater compression to be achieved, by representing transform coefficients with only the minimal precision required to obtain the desired level of image quality. Quantization of transform coefficients is one of the primary sources of information loss in the coding path. In the decoder the dequantization stage tries to undo the effects of quantization.

Entropy encoder and decoder – After quantization is applied, the resulting data are passed to entropy encoder which compresses the data into an organized file containing

headers and markers for each sub-band and more information regarding ROI. The decoder decompresses data of the needed sub-bands and passes it to dequantization.

2. Project goals

Find a way to rescale an image using interpolation. The rescaling should be done during the decoding process, and not by maintaining a large image buffer and apply interpolation afterwards. The rescaling process should consume a little as possible memory bits.

As illustrated in Fig 4, these goals are obtained by applying a low memory algorithm to the inverse intracomponent transform stage (IDWT) which includes sub-band recomposition and image scaling.

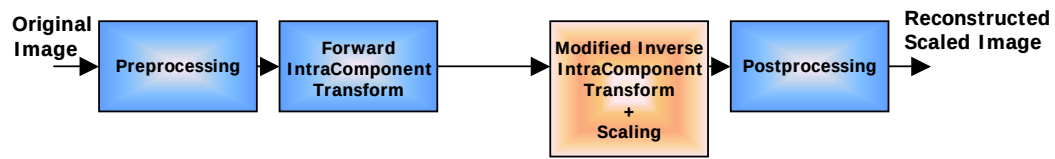


Fig 4: Modifying Inverse Wavelet Transform to obtain image transformation

3. Proposed algorithm

The proposed algorithm consists of cyclic 4 lines buffer for each level of decomposition. Each level when processed uses its own buffer and passes the results to the lower level buffer. The algorithm main stages are described in the following section.

Samples interleaving – In this stage of the algorithm samples from 4 bands of a given level are arranged in 4 lines buffer in the following order $\{LL_n, HL_n, LH_n, HH_n\}$ as illustrated in Fig 5.

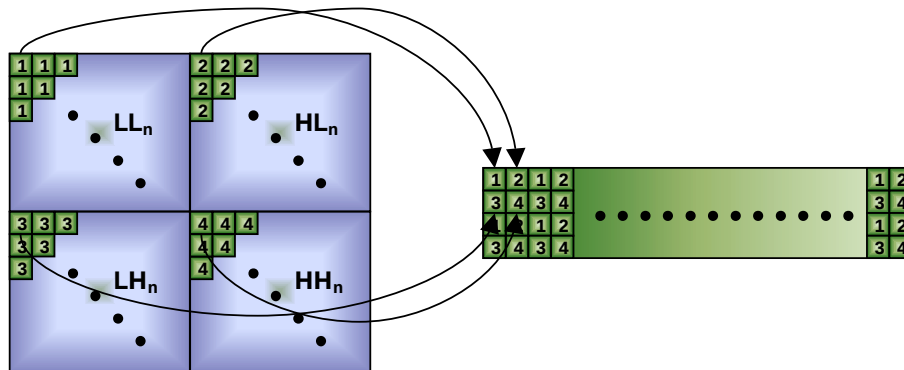


Fig 5: Samples interleaving in 4 lines buffer

Horizontally filtering – This stage takes as an input the organized buffer and filters all lines that are not yet processed. Filtering is done using the lifting realization, which means that even samples use a 3 tap filter and odd samples use a different 3 tap filter. Horizontally filtering is illustrated in Fig 6.

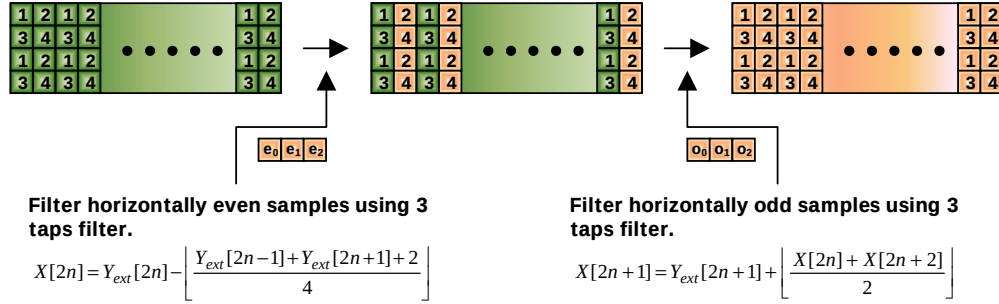


Fig 6: Horizontally filtering

Vertically filtering – This stage is trickier because we can't filter vertically all columns due to the 4 lines buffer. Filtering is done on all rows that are currently in the cyclic buffer, and will continue when new samples will be arranged.

On Fig 7 we can see that first line 0 is vertically filtered, then new samples are arranged and horizontally filtered, then line 2 is vertically filtered, and finally line 1 is vertically filtered.

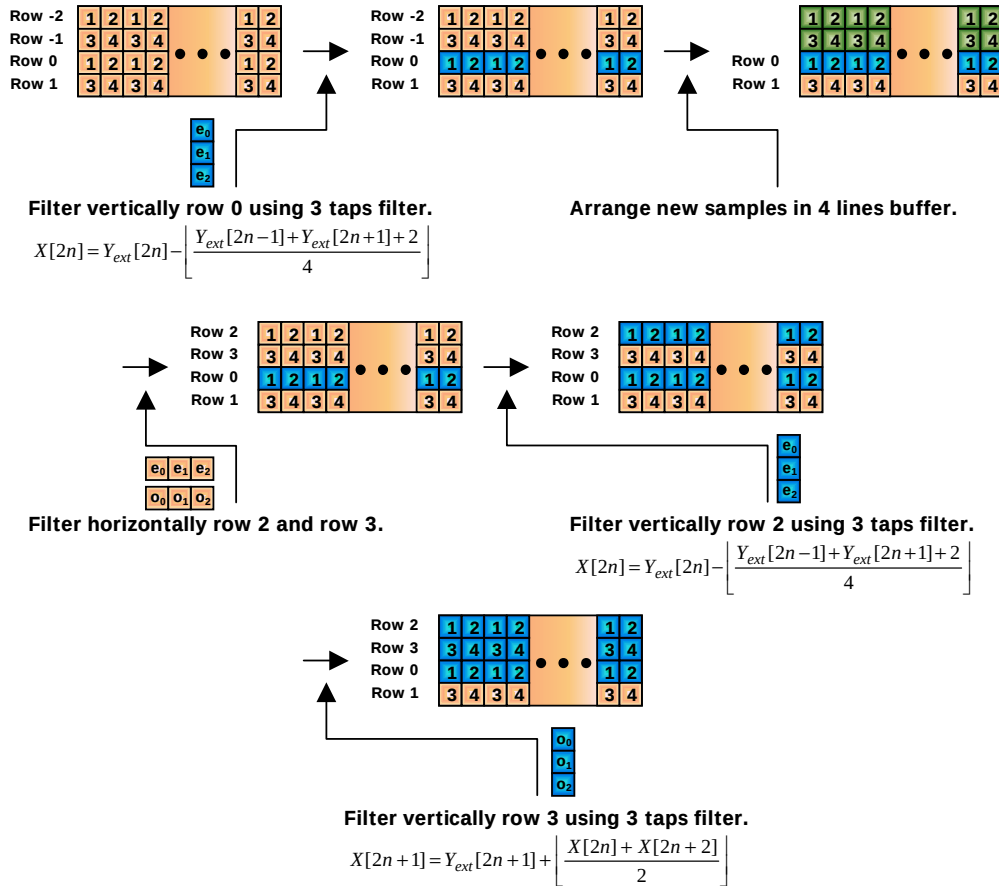


Fig 7: Vertically filtering

Recursive recomposition – 4 lines buffer filtering is done for every level of sub-band. The LL sub-band exists in the compressed file only on the highest level of decomposition. Therefore, to obtain the LL sample, filtering up to the highest level is needed. For example, if LL_0 is needed, LL_1 should first be computed. To compute LL_1 , LL_2 should be computed. Until we reach the highest level which should be in the compressed file. This process is illustrated in Fig 8.

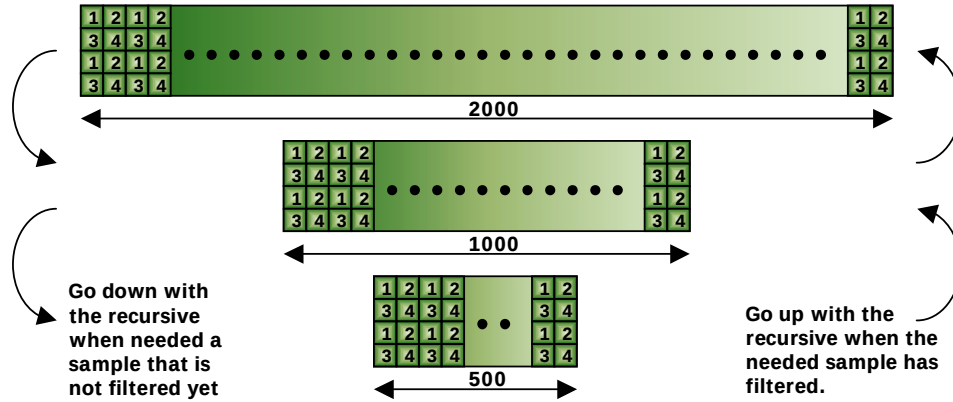


Fig 8: Recursive recomposition, with 2000x2000 image

Image scaling - Scaling is done by simple zero padding. It consists of two stages, the first stage is renaming the existing sub-bands levels and the second stage is adding a virtually level which consist all zeros. An example of 3 levels of sub-bands renaming is shown in Fig 9 and Fig 10.

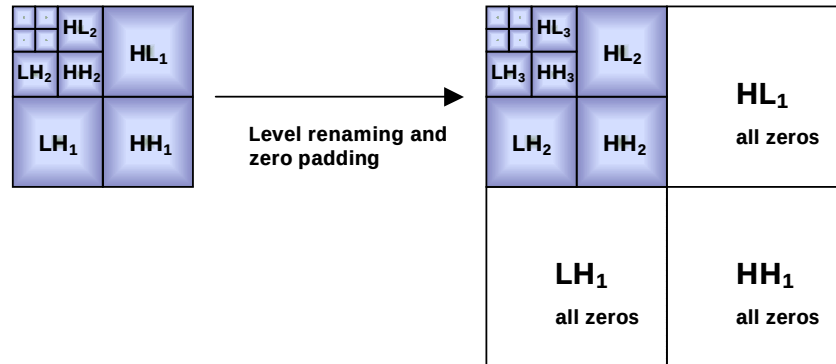


Fig 9: Image scaling using level renaming and zero padding

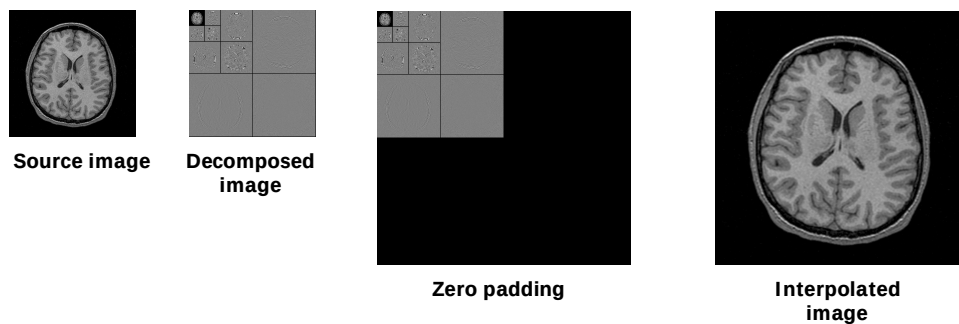


Fig 10: Example of image scaling procedure

4. Memory usage

As described before, the algorithm uses a 4-lines-buffer for each level. Each buffer holds 4 lines of a sub-band level whose length depends on the level. Level 0 line length is the same as the image length. An example of 5 levels of decomposition and image size is 2048x2048 yields the following memory usage.

Level 0: 4 lines x 2048 = 8 Kbyte.

Level 1: 4 lines x 1024 = 4 Kbyte.

Level 2: 4 lines x 512 = 2 Kbyte.

Level 3: 4 lines x 256 = 1 Kbyte.

Level 4: 4 lines x 128 = 1/2 Kbyte.

Total of: 15.5 Kbyte.

In general the memory usage is

$$MemUsage = 16 \cdot sizx \cdot \left(1 - \frac{1}{2^L}\right)$$

Where $sizx$ is the image length and L is the number of levels.

5. Results

Results are measured with respect to an ideally low-pass filtered, down sampled test image. Interpolation is done using several techniques including nearest neighbor, bilinear, bicubic and JPEG-2000 recomposition filters. The RMS error is measured with respect to the original image. As one can see the 9/7 filter of JPEG-2000 gives the best results. Fig 11 shows interpolated images.

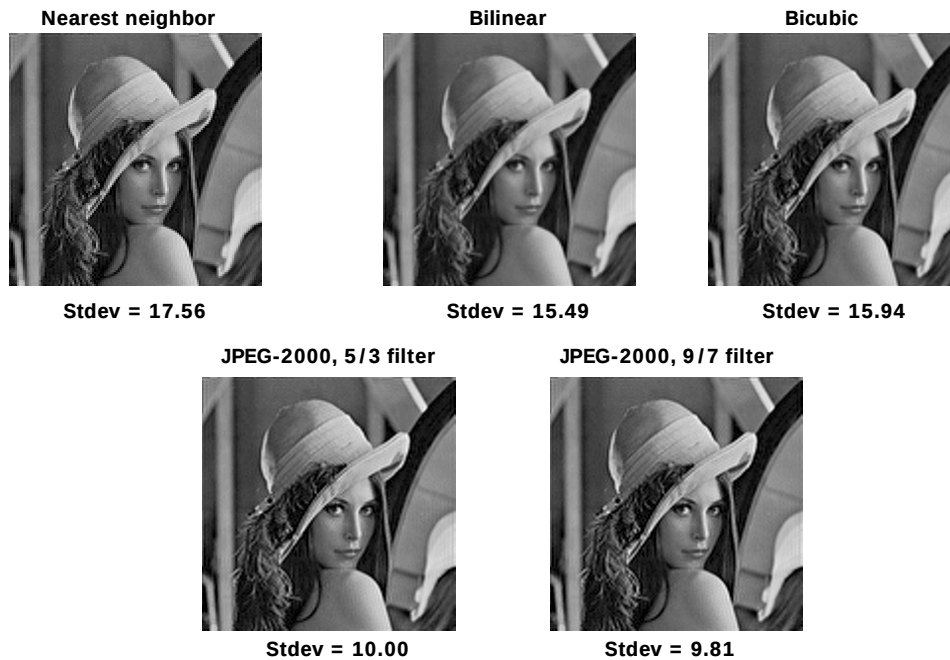


Fig 11: Interpolated images using JPEG-2000 recomposition filters